AgentWorkShop: A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Deploying industrial agents across production scenes requires scene-specific signal and protocol mappings while keeping task context, device dispatch, and resulting evidence aligned. This paper presents AgentWorkShop, a modular task-to-device integration framework that composes task adapters, production context, named operations, semantic input/output, runtime checks, protocol drivers, and service-owned records. Registered task routes pass through the collaboration runtime; direct operation calls enter the shared dispatcher without creating a task record. The design keeps approval, command readback, and subsequent process observations distinct while linking them to the active production context. We evaluated registered task paths and direct operation calls in cast-film and annealing simulators with different signal and protocol mappings. Six sequential model-mediated trials and two fixture-backed command-line tasks preserved their route-specific record links; direct calls retained operation, readback, and run-tagged sample records. Every current-route sample remained below its target band. A mock-state approval-context test refused 12 stale changes before writes, whereas an ablation allowed six product/run changes through. The results demonstrate integration behavior for selected preconfigured paths and a software dispatch guard, not process improvement, configuration reuse, general portability, or comparative platform benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Existing work addresses end-to-end automation control, operator-reviewed control-sequence generation, modular factory skills, and production scheduling [1]–[4]. Other studies target PLC code generation, context-aware control, resource capability discovery, shop-floor coordination, digital-twin fault handling, natural-language machine access, and robotic-line control [5]–[10]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. An integration architecture must accommodate these differences around shared task and operation services while keeping each action tied to its active production context. Recent industrial-agent architectures add AAS-grounded OPC UA actions with signed approval, P&ID-grounded proposal checks, and governed sensor–MES–ERP evidence streams [11]–[13]. These works address asset-specific execution, process validation, or closed-loop analytics. At their software-integration boundary, this paper examines a specific implementation requirement: distinct task adapters and direct tool callers should invoke one shared named operation handler while retaining evidence appropriate to each route. Task routes can link task, approval, command receipt, and process observation; a direct call has operation records but no task edge. This is a scoped implementation contract, not an assumption that prior platforms lack equivalent behavior.

Consider two production scenes with different process signals and interfaces: cast-film thickness control and annealing-zone temperature control. An agent task in either scene needs production context, typed signal access, and a route from a named operation to the configured device protocol. The framework must also distinguish a command receipt from a subsequent process observation. The shared task and operation services can therefore be configured with scene-specific signal maps, protocols, and limits while retaining a common execution path. The evaluation instantiates this path in two simulator scenes.

This paper presents AgentWorkShop as a modular framework for task-to-device integration in industrial agents. It connects collaborative tasks through a registered harness interface to production context, manufacturing execution system (MES) data, analysis and modeling services, data-acquisition (DAQ) and write-control (DCW) nodes, and protocol-specific drivers. Its interfaces include command-line, web, software development kit (SDK), Representational State Transfer (REST), Model Context Protocol (MCP), and Agent-to-Agent (A2A) surfaces. Task and chat routes enter the collaboration runtime, whereas a direct REST tool-invocation route can call the shared operation dispatcher without creating a task. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and agent bindings scope operations. Agents and callers invoke named operations through shared runtime checks; the framework does not choose routes autonomously and retains proposals, command confirmation, and process observations as distinct records. Fig. 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV follows task context through shared dispatch, device admission, approval, and route-specific evidence.

The evaluation asks whether a registered task harness, a fixture-backed command-line task adapter, and direct REST calls reach the same named operation across two configured simulator mappings while preserving the records applicable to each route. A fixed-call replay separates task-path behavior from an external model decision, and a mock-state experiment examines the approval-bound context check before software dispatch. The study therefore tests selected preconfigured routes; it does not test configuration reuse, new-scene onboarding, or general adapter interchangeability. Section V describes the protocol and reports each evidence type separately.

The paper makes two contributions:

1) It implements a modular execution path that brings production context, semantic I/O, operation checks, configured protocol drivers, and service-owned records to one named-operation boundary. Task routes retain

linked task, approval, receipt, and observation records where available; direct calls retain operation records without creating a task. For approved recipe actions, the system binds approval to the active line–product–recipe–run tuple and checks it again after asynchronous authorization, just before software dispatch. This software check is not an atomic lock across device I/O, a new protocol, a safety-verification method, or a process-control algorithm.

2) It reports a bounded implementation case study of route-specific record continuity for two registered task adapters and direct REST calls across cast-film and annealing simulator mappings, together with a mock-state test and ablation of the context check. The results cover selected preconfigured paths; they do not establish process attainment, configuration reuse, general adapter portability, or comparative platform value.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production and automation abstractions for context, tool access, and operational constraints. Table I gives a focused, nonexhaustive comparison of features explicitly described in selected studies of industrial agents' production context, plant-facing operations, or action, approval, and evidence pathways. "Not specified" means that the cited study did not describe the feature; it does not imply that the feature is absent. Evaluation descriptions are reported at the level given by each source, without normalizing metrics or success criteria across studies.

A. Industrial Agents and Plant Integration

Industrial-agent systems, including large language model (LLM)-based systems, span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [1]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [2]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [14]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [5], [6]. Manufacturing studies consider dynamic resource-capability exploration [7], multi-agent shop-floor coordination [8], and digital-twin-supported fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated serial line [4] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

The closest architecture-level comparison is Xia et al., who connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences; their case studies address robotic automation and process simulation [15]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, plan constrained manufacturing steps, or compose semantic recipes from OPC UA skills [3], [16], [17]. These studies describe important parts of plant-facing integration. The recent systems in Table I also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [11]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [12]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control, evaluated on real manufacturing records [13]. These works address asset-specific approval, process validation, or closed-loop analytics. The complementary question in this study is whether a modular task-to-device implementation can route distinct registered task adapters and a direct operation caller through a shared named handler while preserving the records applicable to each route. The implementation composes production context, semantic I/O, configured drivers, command receipts, and later process observations; its approved recipe path also rechecks the line–product–recipe–run tuple before software dispatch. We evaluate this contract as a bounded implementation case study across two preconfigured simulator mappings. The results do not establish configuration reuse, unseen-scene onboarding, reduced integration effort, or superiority over another platform. Table I summarizes the explicitly reported features and evaluation scopes used to position this contribution; it is not a performance comparison.

The operation checks complement safety-aware decision support [18] but do not replace independent process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]; the operation-level scope, policy, grant, and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [23], [24]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. AgentWorkShop

TABLE I
FEATURE-LEVEL COMPARISON OF SELECTED PLANT-FACING INTEGRATION STUDIES. ENTRIES SUMMARIZE REPORTED DESIGN SCOPE AND EVALUATION, NOT PERFORMANCE.

System	Context and operation checks	Device-facing route	Reported evaluation
Xia et al. [15]	Cognitive, twin, and physical layers; snapshots, events, plans	Digital-twin/physical automation	Robotic automation and process-simulation cases
Köhle et al. [3]	Factory agents with injected live state; state-based tool filtering	MCP exposes OPC UA methods; MQTT coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	Constrained LLM planning; operation checks not specified	MCP-exposed manufacturing functions	Laboratory multi-step process
Köhle et al. [17]	AI-assisted recipes from OPC UA skills; checks not specified	OPC UA skills and virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [11]	AAS-grounded context; signed human approval	PLC-validated OPC UA Methods; immutable audit log	Two laboratory PoCs; single scripted runs
Schall [12]	P&ID, SCADA, time-series, SOP; topology and rule checks	Advisory proposal with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
Lin and Chen [13]	Sensor–MES–ERP thread; SOP-, risk-, capability-aware gating	Edge monitoring and closed-loop actions	25,275 real records; operational metrics
This work	Registered task-harness routes and direct calls; approval-bound context rechecked before recipe dispatch	Shared dispatcher; semantic nodes resolve to configured DAQ/DCW drivers	Current route and record-link checks across two simulator mappings; deterministic task fixture; no process-attainment claim

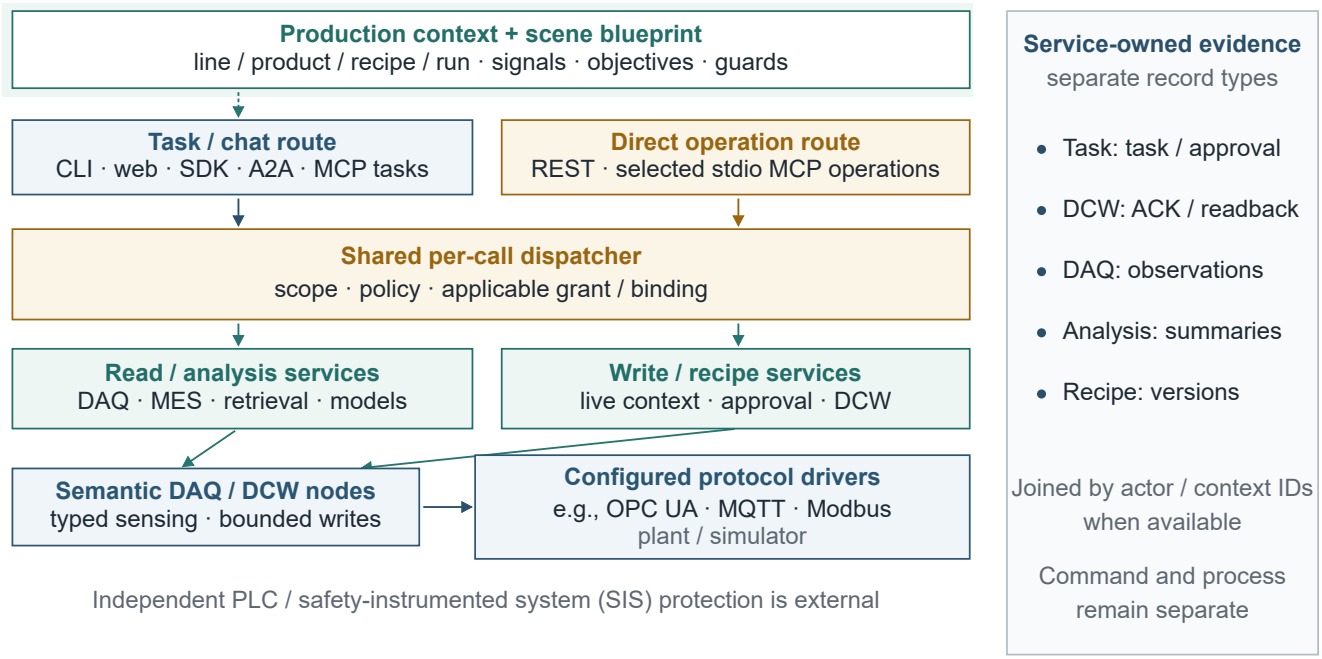


Fig. 1. AgentWorkShop architecture and record ownership. Task/chat calls pass through the listed access surfaces and a registered harness; direct REST and selected stdio-MCP operation calls enter the shared dispatcher, while MCP management/task methods use their service routes. The evaluation exercises direct REST operation calls; the stdio-MCP route is shown but not tested. Recipe writes recheck the approved line–product–recipe–run tuple; each service retains its own records. Independent programmable logic controller and safety-instrumented system protection remains external.

composes such abstractions with its task runtime, operation checks, semantic I/O, and route-specific records; this paper evaluates selected paths through that composition rather than proposing a replacement for the underlying standards.

III. System Design

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—organize six design rules (D1–D6). These are deployment constraints drawn from the target workflows. D1 stores scene settings as versioned data; D2 provides registered harness, tool, and driver interfaces; D3 separates proposal from execution; D4 treats the tool catalog as discovery rather than permission; D5 routes actuator writes through the common DCW admission point; and D6 retains evidence by operation.

The requirements trace to implementation as follows. Scene variability maps to D1, scene blueprints, and context resolution; harness variability and extensibility map to D2, the registered harness and driver interfaces; operation authority maps to D3–D5, the shared dispatcher, grants, bindings, and write admission; outcome observability maps to D6, route-specific receipts, and observations. These interfaces are design extension points; the evaluation in Section V tests selected registered-harness and direct-operation routes, not general harness interchangeability. AgentWorkShop supplies task and operation services; agent strategy and independent process protection remain outside this design.

B. Layered Architecture

Fig. 1 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. Direct REST tool calls and the stdio MCP operation bridge can call the shared dispatcher without creating a task or invoking a harness; MCP management and task methods remain on their service routes. Direct calls have no agent-turn prompt context, so handlers resolve current route state from the request and services at invocation. The governance band projects an operation catalog and checks each invocation. The context band assembles bound-line and active-run information for prompt grounding, while operation handlers resolve the live state relevant to their route. The service band provides acquisition, write control, analysis, retrieval, and optional model services. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Task/chat routes enter the task runtime; direct operation routes enter the dispatcher; and both use shared operation checks where applicable. A harness adapter supplies agent-specific interaction on task paths, semantic nodes hide protocol details from operations, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to available actor and production context. No single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority, context, and applicable limits before dispatch.

Fig. 3 gives reference pseudocode for the registered-harness task path, not one literal orchestrator function. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Section III is implemented through three cooperating functions: task and context assembly, shared operation dispatch, and controlled device access with route-specific evidence. This section follows one agent action through those functions rather than cataloguing every service. The registered interfaces provide extension points; the evaluation tests selected routes and does not establish general adapter interchangeability.

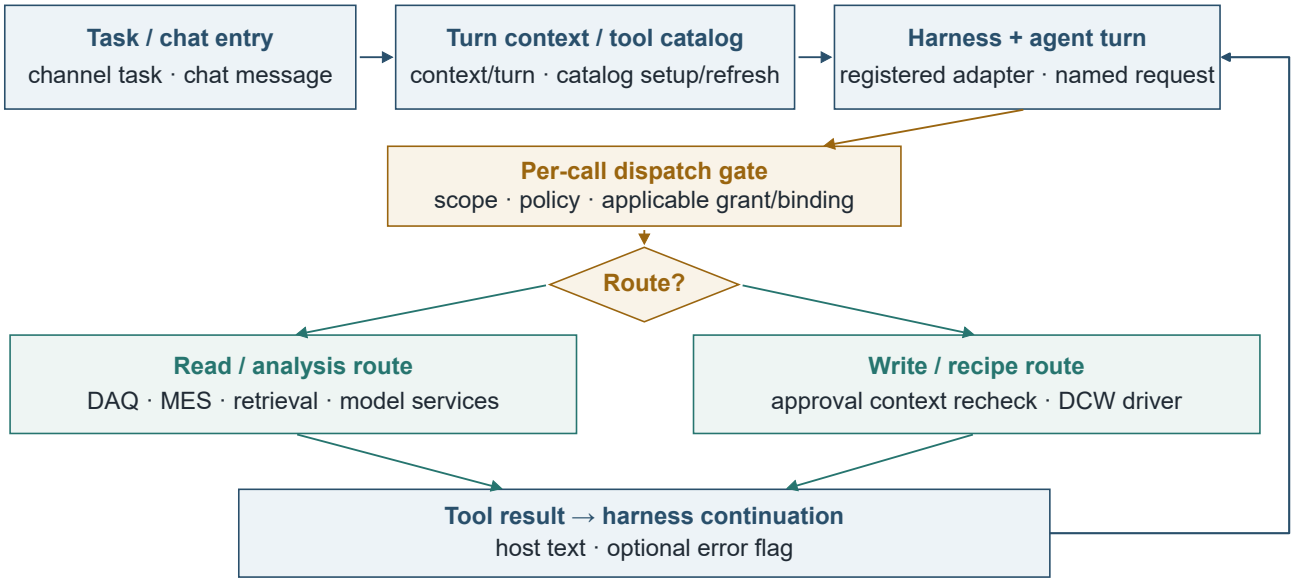


Fig. 2. Registered-harness task flow. Context is assembled per turn; the scoped tool catalog is loaded at setup or refresh. The dispatcher checks scope, policy, and applicable grants and bindings, then returns each tool result to the active harness turn. Direct REST and selected stdio-MCP operations enter the shared dispatcher without a task. In the model-mediated task experiment, the runner performed the follow-up probe, ingestion, and run-tagged query outside the agent task. A command receipt remains distinct from a process observation.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    $\textsc{EmitToolErrorToHarness}$(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckGrantBindingOwnership}(a, u)$
    if not v **then**
      $\textsc{EmitToolErrorToHarness}$(h,u)
      continue
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  $\textsc{EmitDeclaredToolResultToHarness}$(h,r)
end for
return task state and tool results

Fig. 3. Reference pseudocode for the registered-harness task path. Direct operation invocation is not shown. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, task state, and shared execution context; a harness registry adapts each supported agent engine to a common execution interface. At each agent turn, the prompt builder assembles the task, active scene description, production context, channel profile, agent binding, and retrieved memory. This keeps engine-specific interaction behind the harness interface while preserving the current line and operation context for the agent.

Direct REST and selected stdio-MCP operation calls can enter the same dispatcher without a task or harness turn; management and task methods use their respective services. A direct call has no prompt context, so its handler resolves live route state from request identifiers and service data at invocation. For either entry path, prompt context and tool-catalog visibility describe available information and capabilities; neither grants write authority.

B. Shared Dispatch and Device Admission

The dispatcher is the shared operation boundary. It checks invocation scope and current channel policy, then verifies the agent binding, runtime grant, and ownership of referenced resources before selecting a handler. Handlers revalidate route-specific state at execution time; a previously displayed tool catalog does not preserve authority after a policy or grant changes. Scene blueprints define device and signal maps, objectives, and process guards. Capability profiles and tool policies narrow available operations, while bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are

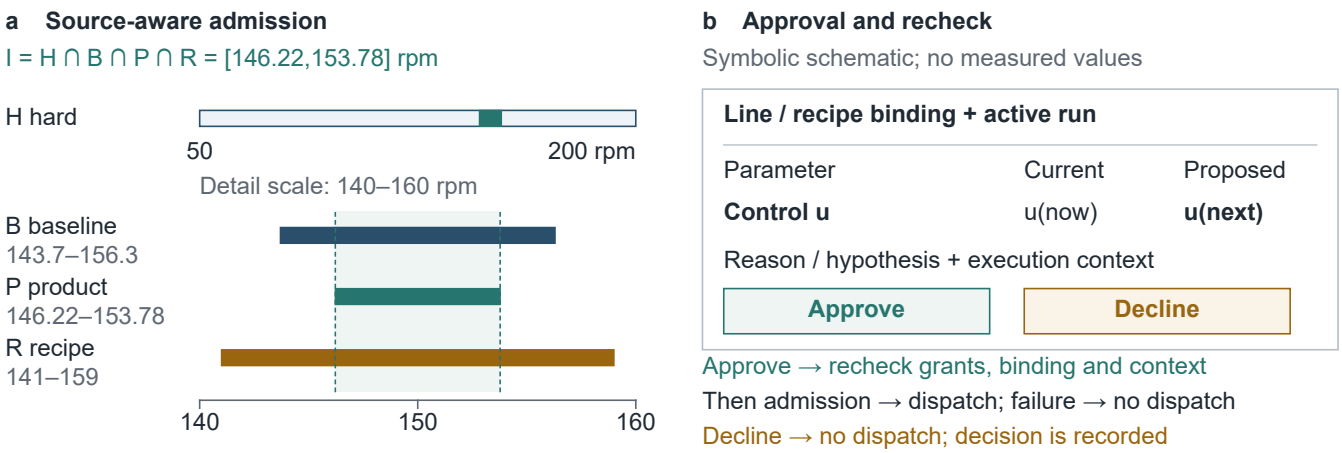


Fig. 4. Admission and approval boundaries. (a) Illustrative archived RPM configuration, not experimental data: online node interventions use the four-layer interval of Eq. (2), whereas recipe dispatch omits the previous recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits. Approval does not reserve equipment state.

invoked through registered handlers; the framework does not prescribe a single optimization strategy.

Actuator writes converge on the shared write-control (DCW) admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction; an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks also constrain each route. These software checks bound commands but do not replace independent programmable-logic-controller or safety-instrumented protection.

C. Approval, Execution, and Evidence

Routes add timing and approval checks appropriate to their operation. Online node writes enforce a 60 s minimum spacing; recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. The default approval wait expires after 180 s. A changed recipe definition requires a new `recipe_apply` proposal. These checks reject stale requests at the software dispatch boundary; they do not lock equipment state across network or device I/O.

Fig. 4(a) illustrates the four-layer value interval in Eq. (2) using an archived RPM configuration. Panel (b) shows a symbolic approval followed by authority and context rechecks; its values are illustrative, not experimental measurements.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller obtains later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. DAQ stores subsequent timestamped process observations, while recipe versions, approvals, and task records remain service-owned records linked where a route supplies the corresponding identifiers. A verified setpoint confirms the command path, not target attainment. This distinction defines the evidence chain evaluated in Section V.

V. Experimental Evaluation

The experiments examine three distinct questions: whether selected task and direct-call routes preserve their required records across two configured simulator scenes; whether an approval-bound context check blocks stale recipe actions before software dispatch; and what process traces were recorded in a separate historical campaign. These evidence types are reported separately. All plant-facing tests used software simulators; no physical plant or independent process replication was involved.

A. Questions and Protocol

For a registered task route, the primary integration criterion is a completed task with an approval, a readback-verified command receipt, and a process sample tagged to the active run and linked to the task and approval. The fixed-call replay is assessed for record links but not task completion. A direct operation call has no task edge; its criterion is a verified receipt and readback together with a sample tagged to the active

run. The context guard is assessed separately by whether line, product, recipe, or run changes made while approval is pending are refused before a driver write.

The current-version route study used the v0.8.0 application with cast-film and continuous-annealing simulator scenes. In cast film, the operation wrote the screw-speed setpoint through OPC UA and observed thickness through MQTT; the target band was 48–52 μm. In annealing, it wrote the zone-3 temperature setpoint and observed strip temperature through Modbus TCP; the target band was 700–760 °C. Each scene used one persistent simulator instance. The model-mediated registered-harness path performed three sequential trials per scene. Each trial had a separate logical line, product, recipe, run, channel, agent, and task, while process state carried forward within the scene simulator.

For each model-mediated trial, a candidate from a prevalidated set was submitted as a `recipe_trial` and received synthetic administrator approval. The handler returned a readback-verified receipt. After 20 s, the runner probed and ingested one sample, then queried it by active-run ID. The isolated MQTT and Timescale endpoints were unavailable in this run, so the sample query used a local SQLite fallback. This tests the selected application record path, not normal online DAQ operation or process response. Separate checks exercised a fixed-call harness replay, a deterministic fixture-backed command-line task adapter, and direct REST operation calls. The harness and command-line fixtures did not use an external model decision; the direct REST route created no task.

The context-guard suite changed one field of the approved line–product–recipe–run tuple during approval in twelve cases. A separate fault-injection test for `recipe_apply` queued a context mutation after the grant-revalidation helper returned but before the caller's final synchronous pre-dispatch comparison. The caller refused the request before dispatch. A test-only ablation removed the tuple comparison while retaining authority, range, and other route checks. Outcomes were recorded in mock-driver state. Assertion counts below describe software checks, not independent process samples or reliability estimates.

Each registered-task trial followed the same execution chain. First, the runner created a task in a configured simulator scene and resolved its active line, product, recipe, run, channel, and bound operation. Second, the registered harness submitted a candidate setpoint for synthetic administrator approval. Before dispatch, the recipe handler rechecked operation scope, parameter limits, and the approved production context. Third, the shared dispatcher invoked the scene's configured protocol driver and returned a receipt only after the requested value had a matching readback. Fourth, the runner waited 20 s, probed one process variable, tagged the observation with the active run, and queried the task, approval, receipt, and sample records. Because the isolated MQTT and Timescale endpoints were unavailable, the sample query used a local SQLite fallback. This is the tested execution and record path; the guard mutations were injected in separate mock-state tests. The fixed-call and command-line routes supplied deterministic fixture input, whereas direct REST entered the shared operation boundary without creating a task record.

TABLE II
CURRENT-VERSION ROUTE AND CONTEXT-GUARD CHECKS. ASSERTION COUNTS ARE SOFTWARE CHECKS, NOT INDEPENDENT PROCESS REPLICATIONS.

Execution path	Result and inference limit
Registered task harness	Six sequential tasks; 6/6 task-to-sample joins and verified receipts; 76 runner assertions. One sample per task; all below target.
Fixed-call harness replay	One task per scene; 2/2 approval/receipt/sample links; 31 assertions. Task completion was not asserted.
Direct REST operation	One call per scene; 2/2 verified receipts/readbacks and run-tagged samples; 19 checks. No task edge.
Fixture-backed command-line task	One completed task per scene; 2/2 task-to-sample links; 37 application and 28 isolation checks. No external model decision.
Approval-context guard	12/12 injected tuple changes blocked before mock writes; one queued between-check mutation was refused before dispatch. Mock state only.

B. Current-Version Route-Integration Results

All six tasks completed on the model-mediated registered harness and passed the task, approval, receipt, and sample-link checks (76 runner assertions). Cast-film settings of 125/130/125 rpm produced post-trial samples of 45.69/46.69/45.69 μm. Annealing settings of 745/745/745 °C produced samples of 679.60/677.70/678.10 °C. All six samples were below their respective target bands. Each was a single sample taken 20 s after a trial, so these readings do not describe a response trajectory, steady state, or the quality of the model's choices.

The fixed-call harness replay linked both tasks to approval, receipt, and active-run samples (31 runner assertions) but did not assert task completion. Direct REST returned verified receipts and readbacks with run-tagged samples in both scenes (19 checks); the samples were 46.37 μm and 675.20 °C, both below target. The fixture-backed command-line adapter completed both tasks and linked their samples (37 application and 28 isolation checks); its samples were 46.38 μm and 674.40 °C, also below target. Together, these runs show that the tested task routes and direct operation route used the shared services under two preconfigured simulator mappings. They do not test configuration reuse, unseen-scene onboarding, arbitrary adapter interchangeability, or process attainment.

C. Approval-Context Guard and Ablation

The production handler blocked twelve one-field changes to the approved tuple (three each to line, product, recipe, and run) before a mock write or execution record. A queued context mutation between grant revalidation and the caller's final synchronous check was also blocked before `recipe_apply` dispatch. In the test-only ablation, removing the tuple comparison while retaining other checks allowed six product/run changes through; each produced two mock writes and one execution record. The other six line/recipe changes remained blocked. Across seven suites, 566/566 local unit tests passed, including the targeted approval tests.

This ablation isolates the tuple comparison's effect within the tested software state: it blocks the product/run changes

that the retained checks alone did not reject. It does not demonstrate atomicity across network or device I/O, safe physical operation, operator performance, or end-to-end DAQ integrity. The test count is not a failure-rate estimate.

D. Historical Multi-Scenario Control and Objective Traces

Fig. 5 pairs the control requests with their scene-specific objective variables for four archived simulator scenarios: injection molding, wastewater treatment, continuous annealing, and biaxially oriented polyethylene terephthalate (BOPET) film. The upper panels show the recorded process variable and its target window. The lower panels show each requested control's percentage change from its own baseline,

$$\Delta u_{j,k}^{\mathrm{req}} = 100\,\frac{u_{j,k}^{\mathrm{req}} - u_{j,0}^{\mathrm{req}}}{u_{j,0}^{\mathrm{req}}}, \tag{3}$$

where $u_{j,k}^{\mathrm{req}}$ is the requested value of control j after event k, and $u_{j,0}^{\mathrm{req}}$ is its archived baseline. The event index is the recorded control order, not elapsed time; the step curves carry the last request forward and do not interpolate a process response. The archive contains one scripted seed-42 trajectory per scenario. It does not provide a common scalar optimizer score J across these four processes, so the plotted objective variables must not be interpreted as a shared score. Requested settings are intent records, not independent device readbacks.

The archived terminal values were within their individual target windows: 32.17 g for injection molding, 3.504 mg/L dissolved oxygen (DO) for wastewater, 100.08 HV hardness for annealing, and 25.66 μm thickness for BOPET. Injection ended 0.02 g above its lower limit, while BOPET ended 0.04 μm below its upper limit. In annealing, a 176 m/min speed candidate produced 101.3 HV, outside the 89–101 HV window, and was reverted; the terminal measurement is shown separately. For wastewater, the terminal DO value followed later pH and total-phosphorus control requests for which the archive has no matched DO samples. The first annealing temperature step also lacks an optimization-record identifier. These qualifications limit attribution of the historical curves. The traces illustrate the target–control relationship and recorded reversion behavior, but do not establish current-version performance, cross-seed robustness, convergence, or comparative optimizer quality.

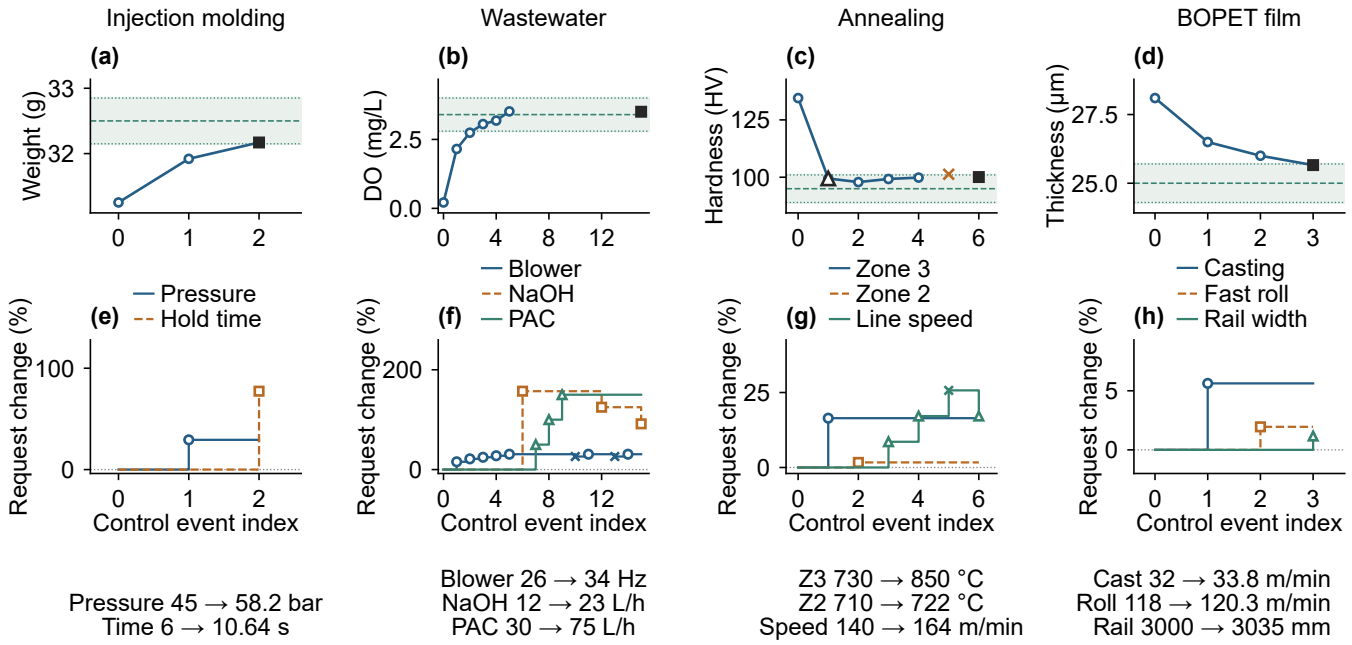


Fig. 5. Historical parameter–objective traces from a seed-42 scripted archive, with one trajectory per process. Panels (a)–(d) show the scenario-specific process variable and target window; panels (e)–(h) show requested control changes relative to each control's baseline. The horizontal axes index recorded events, not physical time, and the step curves carry forward the last request. Crosses mark reverted candidates; the open triangle marks the annealing observation following a temperature step without an optimization-record identifier; filled squares denote separate terminal observations. The wastewater DO trajectory has no matched DO sample after the subsequent pH and total-phosphorus requests. Requested setpoints are not independent device readbacks. These historical traces are separate from the current-version route experiments and do not represent a shared scalar objective.

E. Evidence Synthesis and Limits

The current-version experiments support two bounded conclusions. First, the tested task routes and direct REST calls preserved the specified record associations for their respective route types across two preconfigured simulator mappings, although their single post-trial samples remained below target. Second, the approval-context comparison blocked a defined set of stale-context requests in mock state, with the ablation isolating its effect from retained checks. The historical traces answer a separate descriptive question and must not be combined with the current route results as evidence of process improvement.

The routes test different entry contracts rather than matched control strategies: registered tasks require a task-to-sample edge, while direct REST requires a verified operation receipt and run-tagged sample but has no task edge. For both, the readback check precedes the delayed process sample. This separation makes the result interpretable: an out-of-band sample does not invalidate a verified dispatch or record join, and a complete record join does not imply that the process met its objective. The historical figure adds the missing multi-event context only for the earlier scripted scenarios; it cannot supply a current-version response trajectory.

Supplementary File S1 contains the historical scenario records, current route inputs and logs, available runner and source snapshots, test outputs, and a hash manifest. It identifies which paths can be rerun with deterministic local fixtures. The model-mediated route depends on an external model decision and is not an exact deterministic replay. Each current scene used one simulator instance, registered-harness tasks ran sequentially, and each trial produced one post-trial sample. The evaluation therefore establishes the selected context-to-operation-to-receipt-to-sample links and the tested pre-dispatch guard behavior. The below-target current samples show that preserving this execution chain is not, by itself, evidence of process attainment. The historical figure supplies process-variable and control context, but its older interface and single trajectory per scene cannot validate the current implementation or support independent process performance, physical transfer, configuration reuse, or platform superiority.

VI. Discussion

A. Interpretation

AgentWorkShop composes collaborative-task services, production context, named operations, semantic signal access, configured device drivers, and service-owned records into a shared task-to-device path. The current experiments trace this design through task or direct-operation entry, approval-bound dispatch, readback receipt, and a later run-tagged process sample. Registered task routes preserve the task, approval, receipt, and sample associations; direct REST preserves operation, readback, and sample records without a task edge. The two simulator scenes use different signal and protocol mappings, so the observed joins support the tested composition and route-specific record contract.

The process results bound the interpretation of that evidence: every current-route sample remained below its target band, and the single sample 20 s after a write cannot show convergence or a response curve. The experiments therefore validate selected integration behavior, not process attainment

or improved control. Fig. 5 separately pairs historical control requests with their scenario-specific target variables, including a reverted candidate and terminal values near target limits. This helps explain the intended operation logic—choose a bounded setpoint, observe its process objective, and keep each result attached to its run—but the older single-trajectory archive is descriptive context, not evidence that the current framework reproduces those outcomes.

The context-guard test identifies the software check's contribution under mock state: comparing the approved line–product–recipe–run tuple blocked product/run mutations that the retained checks alone did not reject. This supports a bounded claim about software dispatch. It does not show atomicity across network or device I/O, safe physical operation, or operator effectiveness. Synthetic approval also does not measure human review performance [25]–[27].

B. Scope and Limitations

Only the reported adapter and scene configurations were exercised. The evaluation does not test profile or binding reuse, onboarding of an unseen scene, general adapter interchangeability, normal online DAQ operation in the model-mediated trials, physical transfer, or a platform baseline. The feature-level related-work comparison positions the implementation question; it does not show that AgentWorkShop is more efficient or more capable than another platform. A comparative study would need a shared task set, explicit integration-effort or record-completeness measures, and matched deployments.

The route study used one simulator instance per scene, three sequential model-mediated trials per scene, and one post-trial process sample per trial. These samples are not independent replicates and cannot establish a process response curve. The archived missions likewise provide one trajectory per scene. Follow-up evaluation should vary configuration profiles under a controlled onboarding protocol, use repeated process trials, and compare against a defined alternative integration path. Operation restrictions do not establish security [28]–[30] or replace independent PLC/SIS safeguards [31].

Scope and channel-policy refusals are not uniformly persisted; auditability is limited to the records and denial classes described in Section IV.

VII. Conclusion

AgentWorkShop brings task adapters, production context, named operations, semantic I/O, configured protocol drivers, and service-owned records into a modular task-to-device framework. Tested task routes and direct calls used a shared operation boundary across cast-film and annealing simulator mappings while preserving route-specific records. The current-route samples remained below target, and a mock-state ablation isolated the effect of an approval-bound context check. The study therefore establishes selected integration and software-contract behavior; it does not establish process improvement, configuration reuse, physical transfer, general portability, or comparative platform benefit.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[24] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[25] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[27] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[29] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[30] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[31] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.